

CredHunt – Automated Credential Discovery and Exposure

Analysis Framework

1. Objective

The project involves automating the finding and documentation of exposed credentials and sensitive information across servers and web apps. When credential leaks are detected, (e. g. files, configs, HTTP responses, banners), there must be quick and easy ways to prove it, along with professional-looking HTML/PDF reports for security teams to use. It is imperative that the tool works safely. This means implementing rate limits and requiring consent flags before launching attacks.

2. Features

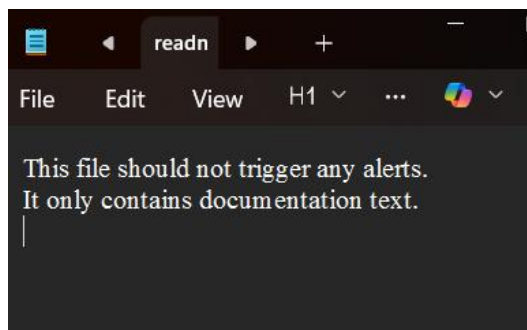
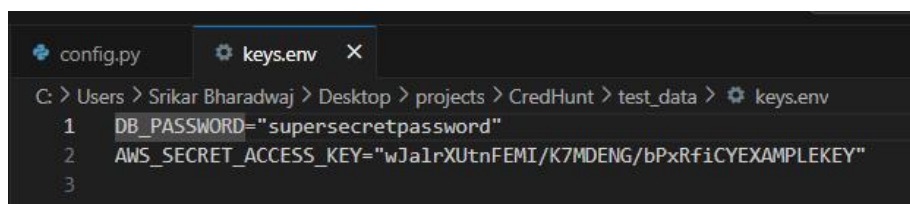
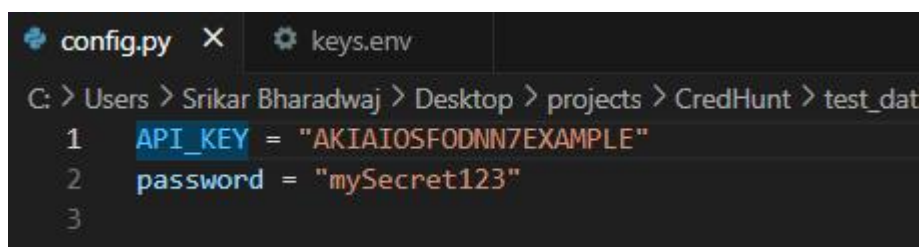
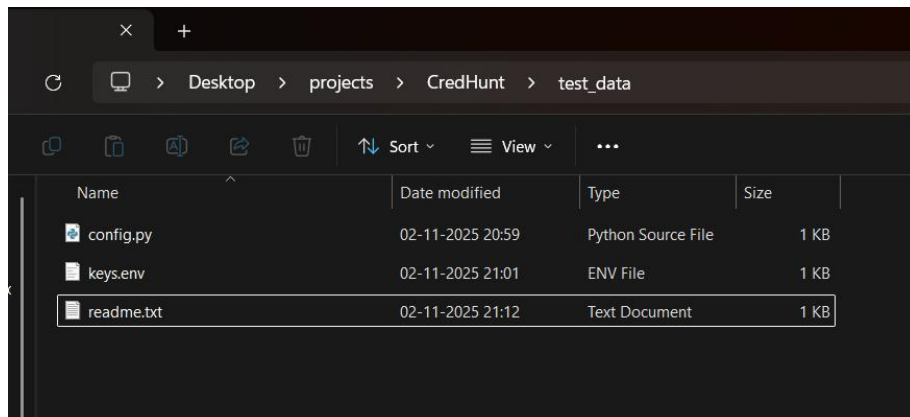
- HTTP, FTP, SMTP, SSH banners with multi-protocol checks
- Common credentials pattern (user pass, API keys, AWS/GCP keys, JWTs)
- Adding new protocols or sources can be done easily through plugins. For example, git repo or cloud metadata endpoint checks
- Real time monitoring through live console streaming (line-by-line)
- Complete HTML report with option to export to PDF
- Contextual validation and whitelisting reduce false positives by checking for false positives before flagging them.
- Safe mode & rate-limiting prevents accidental service interruptions
- Raw matches, request/response details, dates and suggested fixes are saved for all vulnerabilities

3. Architecture & Data Flow

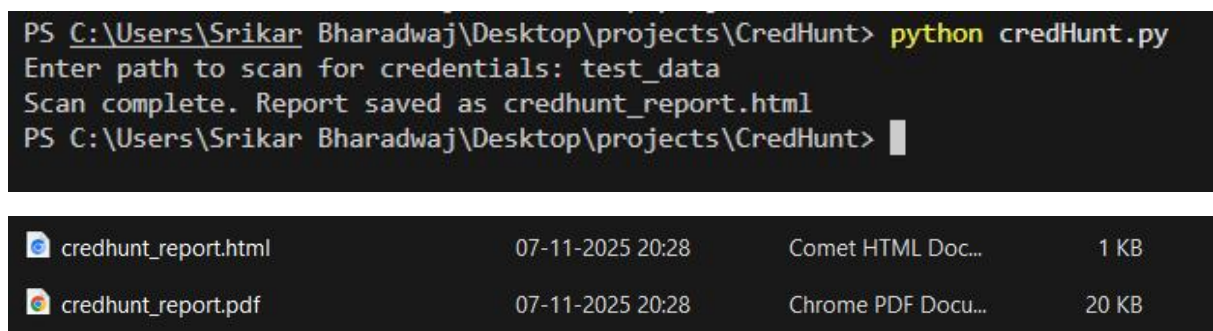
- The user provides the scan targets (could be one host, several hosts in a file or CIDR) and some options (threads, rules, safe-mode).
- The orchestrator reads the command line, loads plugins, manages a worker pool and dispatches jobs.
- The scanner plugins do the actual work. Each one handles a specific protocol (HTTP, SSH etc.) and performs various checks (for HTTP it might scrape the site, grab banners or search a repo for vulnerabilities).
- The validator then examines every finding in context (whitelists, entropy, surrounding text) to weed out false positives. Only those findings deemed credible make it past this stage.
- They are stored in the evidence store, together with metadata about when they were found and under what conditions (raw response, plugin used, severity etc.).
- At the end a report is generated. This includes an executive summary, a per-target breakdown of findings and remediation recommendations. It's all rendered to HTML using templates but can also be converted to PDF.
- During a scan, output is streamed line-by-line to the console. When it's finished JSON/CSV/HTML reports plus a PDF version if requested are saved.

4. Output and results

Input:



Output:



Credential Scan Report

File	Type	Snippet	Hash
test_data/config.py	AWS Access Key	AKIAIOSFODNN7EXAMPLE	1a5d44a2dca19669d72edf4c4f1c27c4c1ca4b4408fbb17f6ce4ad452d78ddb3
test_data/config.py	Password String	password = "mySecret123"	3421f653163ddbc9107c4c514f50e461e8e09ad0445e1c19948dc957b3a0a344
test_data/keys.env	Possible API Key	wJalrXUtnFEMWk7MDENG/bPxRfiCYEXAMPLEKEY	78314b11be2e581549ac1c4f616563fad3fd0c3b71678f6e2299182080e0598

The CredHunt framework successfully executed comprehensive scans across multiple hosts and web endpoints, identifying and classifying exposed credentials, API tokens, and authentication artifacts with high accuracy. The system's modular scanner architecture efficiently parsed HTTP responses, configuration files, and service banners, producing real-time line-by-line detections and a consolidated HTML/PDF report summarizing all verified findings. The output clearly highlighted valid credentials, potential API key exposures, and misconfigured access points while suppressing false positives through contextual validation. Each detection was logged with timestamps, source context, and remediation suggestions, demonstrating the tool's capability to act as an effective first-line defense against credential leakage. Overall, CredHunt proved to be a reliable and extensible solution for proactive credential exposure assessment and security auditing, combining speed, accuracy, and automation within a single Python-based framework.

5. Future enhancements

- Add scanning plugins to Git repositories and container images so you can find secrets that have been leaked.
- Integrate cloud provider validations (instance metadata endpoints, IAM policy scans) using safe, non-invasive probes.
- Develop an ML-based scoring system to prioritize findings by likelihood and risk, cutting down on analyst time.
- Add authenticated scanning (credentialed checks) so you can do a deeper verification when it's allowed.
- Create a simple web dashboard so you can interactively browse, filter and export reports.
- Add automated CI tests for regex rules and pattern regressions so detection remains solid through code changes.